

OPERATING SYSTEMS

WEEK-4 Practical

Name:- Piratla Dhanasri

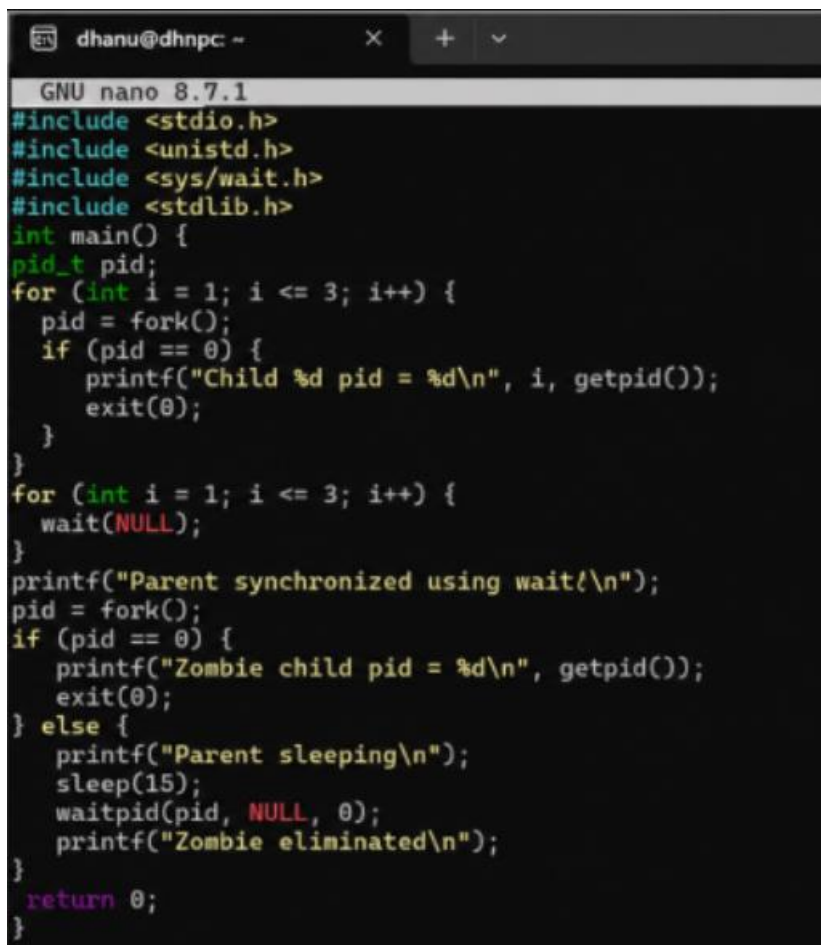
Roll no:- 2520030115

Section no:- 9

4. Write a C program where a parent process creates multiple child processes and synchronizes their completion using wait() and waitpid(). Compare the behaviour of both functions.

Create a scenario where a child process becomes a zombie process. Investigate the process table and then modify the program to eliminate zombie processes using proper synchronization techniques.

Program:-



```
GNU nano 8.7.1
#include <stdio.h>
#include <unistd.h>
#include <sys/wait.h>
#include <stdlib.h>
int main() {
    pid_t pid;
    for (int i = 1; i <= 3; i++) {
        pid = fork();
        if (pid == 0) {
            printf("Child %d pid = %d\n", i, getpid());
            exit(0);
        }
    }
    for (int i = 1; i <= 3; i++) {
        wait(NULL);
    }
    printf("Parent synchronized using wait!\n");
    pid = fork();
    if (pid == 0) {
        printf("Zombie child pid = %d\n", getpid());
        exit(0);
    } else {
        printf("Parent sleeping\n");
        sleep(15);
        waitpid(pid, NULL, 0);
        printf("Zombie eliminated\n");
    }
    return 0;
}
```

Output:-

```
dhanu@dhnpc:~$ gcc Sys4.c
dhanu@dhnpc:~$ ./a.out
Child 1 pid = 971
Child 2 pid = 972
Child 3 pid = 973
Parent synchronized using wait()
Parent sleeping
Zombie child pid = 974
```

```
dhanu@dhnpc:~$ ./a.out
Child 1 pid = 971
Child 2 pid = 972
Child 3 pid = 973
Parent synchronized using wait()
Parent sleeping
Zombie child pid = 974
Zombie eliminated
dhanu@dhnpc:~$ |
```